

Python: get sha256() hash digest of a single bit in Python

Asked 7 years, 4 months ago Modified 1 year, 8 months ago Viewed 4k times

- ▲

1

▼

🔖

🕒
- Or, in general, any arbitrary number or bits that is not a multiple of 8.
- According to [hashlib.py](#) there is one constructor method named for SHA256. Then I use `sha256()` to create a SHA256 hash object. I can now feed this object with arbitrary strings using the `update()` method, and at any point I can ask it for the digest of the concatenation of the strings fed to it so far using the `digest()` or `hexdigest()` methods.
- Well. I would like to feed 1 bit to SHA256 in line with this link ["What is the SHA-256 hash of a single '1' bit?"](#)
- In Python 2.7, of course.
- So, what is the procedure to hash 1-bit long input consisting of the bit "1"? (not the 8-bit long `byte[] { 1 }` input?)

python hash sha256 Edit tags

Share Edit Follow Close Flag Protect

asked Sep 14, 2017 at 11:22

 **DIVVS IVLIVS**
55 ● 2 ● 9

- ▲
- Yes, but the padding is done based on the input's length in bits. There are examples in other languages, there is C code available in [section 8][1] of [RFC 4634][2] to compute the hash of data that is not necessarily a multiple of 8 bits. See the methods whose names are `SHA*FinalBits(...)`. [1]: [tools.ietf.org/html/rfc4634#section-8](#) [2]: [tools.ietf.org/html/rfc4634](#) – **DIVVS IVLIVS** Sep 14, 2017 at 11:48 ✓
- ▲
- yes, BUT THERE IS NO 1-BIT TYPE IN PYTHON – **user3850** Sep 14, 2017 at 11:53
- ▲
- Of course there is, it is called boolean. – **allo** Sep 14, 2017 at 12:07
- ▲
- @allo: care to demonstrate? – **user3850** Sep 14, 2017 at 12:24
- ▲
- @DIUUSIULIUS This sounds like a beginning of an [XY problem](#). Are you trying to achieve something specific, or is it just curiosity? Most real world protocols either pad the number, or have a specific encoding for the arbitrary length integers and hash that instead (like BER). – **viraptor** Sep 14, 2017 at 23:30

4 Answers

Sorted by: Date modified (newest first) ⌵
[Reset to default](#)

- ▲

1

▼

🔖

✓
- You can use <https://pypi.org/project/sha256bit/>
- Same interface as `hashlib` except that the `update` function has an optional argument `bitlen` to support bit granularity.
- ```
>>> from sha256bit import Sha256bit
>>> h=Sha256bit()
>>> h.update(b'\x00', bitlen=1)
>>> h.hexdigest()
'bd4f9e98be68c6ead3243b1b4c7fed75fa4feab1f84795cbd8a98676a2a375'
```
- Share Edit Follow Flag
- edited May 21, 2023 at 9:23
- answered May 19, 2023 at 22:42
-  **acapola**  
1,233 ● 2 ● 13 ● 29
- ▲
- Finally, after 6 years, someone has provided the answer. Thank you! – **DIVVS IVLIVS** Sep 20, 2024 at 13:42

- ▲

2

▼

🔖

🕒
- Neither the `hashlib` API nor the underlying C module support anything but "buffers of bytes".
- Since the SHA standard prescribes adding something to *any* message, regardless of length, you cannot even "pre-pad" your input in python to get around that restriction.
- To demonstrate (taken from <https://www.rfc-editor.org/rfc/rfc4634#section-4.1>):
- ```
>>> sha256('abcde').hexdigest()
'36bbe50ed96841d10443bcb679d6554f0a34b761be67ec9c4a8ad2c0c44ca42c'

>>> sha256('abcde\x80' + 57*'\x00' + '\x28').hexdigest()
'45cb103e6385e1330c892d1566d4d82f0c1c256947e54206704973c6c2ad4f6'
```
- Although calculating the hash of a message with a length that is not a multiple of 8 might technically be allowed, I would very much doubt that there are real world use cases for this (that require the use of only the python standard lib, no less).

Share Edit Follow Flag

edited Oct 7, 2021 at 10:53
 **Community Bot**
1 ● 1

answered Sep 14, 2017 at 18:59
 **user3850**

- ▲

-1

▼

🔖

🕒
-  This post is hidden. It was [deleted](#) 7 years ago by the post author.
- The hash of a single bit `1` for SHA256 is the hash of:
- ```
c0000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000001
```
- To find a hash of bits which don't match a multiple of 64 bytes, you have to pad the message with another `1` bit, then `0`s until the message length is 448 (mod 512). Then you append the original length in the final 64 bits.
- It's described at <https://tools.ietf.org/html/rfc4634#section-4.1>
- So to answer your specific question - there's no popular API which supports messages shorter than octets. You have to do the padding yourself and submit full octets instead.
- Share Edit Follow **Undelete** Flag
- edited Sep 14, 2017 at 23:57
- answered Sep 14, 2017 at 23:44

⬆️ This is wrong. The padding is *always* added. If you input the above "pre-padded" message into hashlib, you will get a incorrect result. I can obviously not demonstrate this with a 1-bit message, but I have added the example from the rfc to my answer. – user3850 Sep 15, 2017 at 18:30

⬆️ @hop You are very correct. I confused the total length L condition with not-complete-block condition. – viraptor Sep 17, 2017 at 9:14

Comments disabled on deleted / locked posts / reviews



0



🔒 This post is hidden. It was [deleted](#) 7 years ago by the post author.

Since SHA pads to block size, it doesn't matter if you input 1-bit, 8-bits or 256-bits. It means that you could use `chr(1)` as input:

```
>>> import hashlib
>>> hashlib.sha256(chr(1).encode('ascii')).hexdigest()
'4bf5122f344554c53bde2ebb8cd2b7e3d1600ad631c385a5d7cce23c7785459a'
```

Share Edit Follow **Undelete** Flag

answered Sep 14, 2017 at 11:36

Comments disabled on deleted / locked posts / reviews